

Project Discovery and Hierarchical Data

Kirill Nevzorov under supervision
of Prof. Christopher Buckley
University of Sussex
School of Engineering and Informatics

April 12, 2026

Abstract

This report presents the design and implementation of an LLM-powered developer assistant built with a Quarkus backend and a web frontend. The central value of the system lies in its workflow: enabling a developer to query project history, explore repository state, and interrogate git artefacts through a natural-language interface rather than through manual inspection of command output. Beyond workflow convenience, the project demonstrates that structuring project knowledge as hierarchically organised package-level data improves modularity and maintainability, making it easier to isolate responsibilities, evolve components incrementally, and reason about the implementation impact of new features across the codebase.

The implementation applies conventional software engineering practices, including CDI-managed dispatch, transaction-scoped persistence, and iterative refinement through staged development and testing. The final build represents a partial realisation of the intended system: Quarkus build-time CDI compilation introduced packaging-stage regressions, and the toolset remained constrained to a minimal working subset. The report therefore presents both the practical strength of the workflow and architecture-level design choices, and an honest account of the remaining gap between the intended design and the delivered artefact.

Contents

1	Introduction	3
1.1	Objectives	3
1.2	Report Structure	3
2	Overview	3
2.1	Coding Agents	4
2.1.1	VS Code Copilot	4
2.1.2	VS Code Copilot with Local Ollama Integration	4
2.1.3	Claude Code	4
2.1.4	Ollama (Model Runtime)	4
2.1.5	LM Studio	4
2.1.6	msty.ai	5
2.2	Vim	5
2.3	Java with Quarkus	5
2.3.1	Features Considered for Implementation	5
2.3.2	Key Libraries	5
2.4	Professional Considerations	6
2.5	Findings and Achievements	6
3	Background	6
3.1	The Role of LLMs in Software Development	6
3.1.1	Key Capabilities of LLMs in Coding	7
3.1.2	Integration with Engineering Workflows	7
3.2	Motivation	7
4	Project Description	7
4.1	Scope	7
4.2	Methodology	8

5	Project Architecture	8
5.1	Application Structure	8
5.2	End-to-End Runtime Flow	8
5.3	Package Map	8
5.3.1	Core Components	9
5.4	Services Layer	10
5.5	Architectural Choices	10
5.5.1	Error and Signal Model	10
5.5.2	Persistence and Context Caching	10
5.5.3	System Integration	10
5.6	AI/LLM Powered Software	11
5.6.1	Understanding Large Language Models	11
5.6.2	Resource Requirements	11
6	Implementation Timeline	11
6.1	Milestone Reference	11
7	Results and Discussion	12
7.1	Key Findings	12
7.2	Discussion	12
7.3	Limitations	13
7.4	Testing and Evaluation	13
7.5	Future Work	13
8	Conclusion	13
9	Appendices	14
9.1	A. System Diagrams and Architecture Charts	14
9.1.1	A.1 Tool Execution Sequence	14
9.2	B. Research Resources and References	14
9.2.1	B.1 Primary Technical Focus Areas	14
9.2.2	B.2 Reuse Method	14
9.3	C. Frontend Workspace Screenshots	15

1 Introduction

In this project, I set out to build an application that interfaces directly with Large Language Models (LLMs), specifically a filesystem exploration assistant powered by locally-run models. The technical goal was paired with a personal engineering objective: to push my limits as a software developer by using LLM-powered agentic coding as a core development method rather than as a peripheral convenience.

My interest in local LLMs grew from hands-on experimentation on my gaming computer, where I quickly saw their practical capability. Instead of treating that experience as a one-off experiment, I used it as the starting point for a structured software engineering project that combines application feature delivery with methodological evaluation.

To support this work in a realistic environment, I built a local inference stack rather than relying on external hosted APIs. I installed Ollama to run a local AI server and invested in two NVIDIA RTX 3060 graphics cards, each with 12GB of VRAM, providing a combined 24GB memory budget for model execution.

This setup enabled me to evaluate a full dual-role workflow: LLMs as part of the application runtime, and LLMs as an active engineering assistant during design, implementation, and refinement. The report therefore examines both what was built and how this agentic development approach performed under practical constraints.

1.1 Objectives

The project was originally scoped to develop a hardware resource governor capable of monitoring LLM server resource utilisation in real time. On closer examination, however, this design proved both redundant and unnecessarily complex: the functionality it would have provided closely mirrors what the host operating system already exposes through standard process and memory management interfaces. I therefore revised the scope to focus on the application layer, implementing instead an agentic web application with capabilities for filesystem exploration and file analysis.

This project is designed to explore the integration of LLMs into software architecture through a practical development process. The work combines implementation and analysis: I developed an LLM-integrated web application while examining how LLM-assisted development practices can be applied in day-to-day engineering work.

The objectives are structured as two interlocking strands. The first strand is technical delivery, specifically producing a working, deployable web application that interacts with LLM models. The second strand is evaluation, documenting and characterising the agentic coding workflow.

Building on these strands, the study also evaluates the quality and maintainability of LLM-generated code relative to developer intent, and reflects on the continuing role of traditional software engineering discipline when substantial parts of implementation are delegated to an AI system. Together, these objectives provide a focused basis for analysing both the benefits and the constraints of LLM use in practical engineering workflows.

1.2 Report Structure

This report comprises three phases: foundational context, design planning, and outcomes. The Introduction frames motivation and the value proposition of querying project state through natural language. Background contextualises LLMs and the chosen project toolset. Project Description outlines the problem domain, functional requirements, and intended capabilities. Project Architecture describes the layered backend, frontend signal-based orchestration, persistence strategies, and planned features. Project Requirements consolidates functional and non-functional constraints.

Implementation and outcome sections ground design in practice. Engineering at Home documents environmental constraints of homelab development and their implications. Implementation Timeline provides a Gantt overview of planned development phases. Results and Discussion present observed outcomes, test results, and gaps between intended design and delivered artefact, assessing what was proven and what remains constrained by packaging limitations or incomplete features.

2 Overview

This section introduces the core technologies used in the project and clarifies how they were combined into a practical development workflow. The setup is presented in two parts: the developer environment and the LLM environment.

The developer environment is centered on Visual Studio Code, with Debian running under WSL as the primary host context for day-to-day implementation work. Developer environment and tooling is managed through Docker with devcontainers to keep dependencies consistent and reproducible across sessions.

Ollama, LM Studio, and msty.ai model servers on the homelab workstation were evaluated for integration with editor-based assistants including VS Code Copilot and Claude Code. These tools proved essential for rapidly developing features and estimating feasibility within available project time.

Heavy use of terminal scripting combined with Vim-style keybindings and Makefile workflow served as an additional productivity factor in this setup within Visual Studio Code. These made the workflow strongly keyboard-driven: routine tasks such as running builds, restructuring files, searching project-wide text, and iterating on report output could be executed with minimal context switching. In practice, this interaction style reduced friction during frequent edit-build-review cycles and supported faster iteration when coordinating code changes, tooling commands, and documentation updates.

2.1 Coding Agents

This subsection compares the coding-agent and local model tooling used during implementation, with attention to practical developer ergonomics, task fit, and observed operational constraints.

2.1.1 VS Code Copilot

VS Code Copilot served as the primary development assistant in this project. It was used extensively to accelerate implementation tasks that would otherwise require repeated documentation cross-referencing, and it generally provided a comfortable and reasonably ergonomic workflow with usable default keybindings and acceptable agentic guardrails. Typical delegated work included rapid frontend React development, coordinated API updates across backend and frontend components, and ongoing documentation maintenance through markdown-oriented subagent workflows. In practice, the available token budget was a material constraint: the free tier was insufficient.

An explorative build-rewrite-analyse workflow implementation required the Copilot Pro Plus subscription.

2.1.2 VS Code Copilot with Local Ollama Integration

VS Code Copilot connected to a local Ollama backend was evaluated for the same classes of tasks as standard Copilot usage. Although this setup reduced direct subscription cost, it depended on powering an auxiliary workstation for each session and did not provide adequate responsiveness for routine development. Once the higher Copilot subscription tier was adopted, the local integration path became functionally redundant for day-to-day work.

2.1.3 Claude Code

Claude Code was tested through a local configuration approach in which the base URL was manually redirected to an Ollama instance. This setup was explored primarily for personal productivity tasks, including hardware-inventory maintenance and draft eBay listing generation, rather than core project delivery. The main observed limitation was context-window pressure relative to available workstation resources, which restricted usefulness despite a polished terminal interface.

2.1.4 Ollama (Model Runtime)

Ollama functioned as a central model runtime for the application and, in some sessions, for agentic coding support. Model selection converged on Gemma4 because it offered a favorable speed profile while still supporting a large context window and CPU plus system-RAM offloading. Other attempted models included multiple Qwen3 sizes, glm-4.7-flash, gpt-oss, Qwen2.5-coder, Llama3.2, and Llama3.1. A consistent practical finding was that local inference quality and throughput depended strongly on careful model selection; a longer project timeline would likely have enabled a more rigorous benchmarking environment for direct model-to-model comparison.

2.1.5 LM Studio

LM Studio was most effective as a question-answering interface in this workflow. Its development-server port mode, intended to expose the bundled model runner over the network, was constrained by binding behavior to localhost:11434, which prevented connections from the devcontainer and remote machines. Relative to msty.ai, its folder and conversation organization workflow was easier to use, though less feature-rich.

2.1.6 msty.ai

msty.ai was also used primarily for question answering. It provided support for Ollama and llama.cpp, with explicit backend options across different GPU vendors and CPU execution. In this project context, these backend controls were not essential because the dedicated Ollama server already handled automatic GPU/CPU layer offloading across available processing units. Compared with other interfaces, msty.ai offered broad capability but a noisier, less developer-ergonomic user experience.

2.2 Vim

Vim-style editing was central to this workflow, supported by custom keybindings: **Ctrl-A** (rebind of **Shift-A**), **Ctrl-B** (terminal management), **Ctrl-G** (file management), and **Ctrl-I/Alt-I** (agent management). These mappings made routine typing and editing operations significantly more fluid and reduced interruption during implementation.

VS Code Copilot also integrated effectively as inline autocomplete. Useful suggestions and refactoring actions were commonly accepted directly with a **Tab** keypress, avoiding repeated navigation to the refactoring menu. However, suggestions occasionally lagged behind developer input, despite offering faster iteration than manual refactoring approaches.

2.3 Java with Quarkus

2.3.1 Features Considered for Implementation

At the start of this project, I expected to explore a wider implementation space than what would eventually fit into the final report. I planned for support of multiple LLM providers, including internal Testcontainers-based options and external Ollama base-url based options, so that the system could stay usable under different runtime conditions. I also wanted to keep a native deployment path available for lower-footprint execution and to include workstation-oriented integration testing for real model and tool interactions.

During development I explored different higher-level usability goals, including nested menu and assistant session flow through declarative coding, clean entry and exit behavior, and predictable returns to parent contexts. In parallel, I focused on seeding the frontend codebase with strong design decisions, including packaging the Web UI as a React application with modules and a signals library implemented as separate logical packages.

2.3.2 Key Libraries

Quarkus REST (quarkus-rest): for building API endpoints.

SmallRye Config: for runtime configuration.

Panache ORM (quarkus-hibernate-orm-panache): for persistence-layer development.

LangChain4j (quarkus-langchain4j-ollama, langchain4j-ollama, langchain4j-http-client-jdk): for LLM integration and orchestration.

Picocli (quarkus-picocli): for command-line interfaces.

JLine: for interactive terminal behavior.

Quarkus REST Client (quarkus-rest-client): for outbound HTTP calls.

Quarkus WebSockets Next (quarkus-websockets-next): for websocket streaming.

Quarkus REST Jackson (quarkus-rest-jackson): for JSON serialization.

Quarkus Micrometer (quarkus-micrometer): for telemetry and metrics.

Quarkus Arc (quarkus-arc): for dependency injection.

Quarkus Agroal + SQLite JDBC (quarkus-agroal, quarkus-jdbc-sqlite): for database connectivity and persistence.

Quarkus Quinoa (quarkus-quinoa): for frontend build integration.

Testing stack (quarkus-junit, maven-surefire-plugin, maven-failsafe-plugin): for unit and integration tests.

These libraries formed a practical foundation for turning planned capabilities into working features.

2.4 Professional Considerations

I have developed this project by applying the knowledge and skills acquired throughout the Computer Science modules (G400) taught at the University of Sussex. In considering my professional responsibilities, I have used the [BCS Code of Conduct](#) as a supporting framework. In particular, I have sought to ensure that any evaluative claims made about software systems are presented carefully and responsibly, consistent with section 2.f, which requires members to avoid injuring others, their property, reputation, or employment through false, malicious, or negligent action or inaction.

This research does not involve the collection of user data or other sensitive information, because it is based on the analysis of software and related materials that are already in the public domain. The discussion is intended to contribute constructively to professional practice, in line with section 4.e of the BCS Code of Conduct, which states that members should encourage and support fellow members in their professional development. This also aligns with the Code's wider expectation that continuing professional development should broaden knowledge of the IT profession and help maintain professional competence.

2.5 Findings and Achievements

This project pursued higher-order workflow capabilities through a specification-as-documentation approach rather than isolated endpoint implementations. These included a menu-driven interaction model for assistant sessions, modular tool orchestration across exploration, reading, writing, introspection, and persistent project knowledge, and Retrieval-Augmented Generation with reproducible results.

The application stack supports practical reliability and governance features, including transactional execution boundaries, explicit error pathways, telemetry-backed observability, and compatibility handling for evolving tool interfaces.

On the frontend, it supports progressive frontend integration for browsing, summarisation, and activity tracing, with discoverability and UX gaps easier to identify and address iteratively. Overall, a platform that could be refined continuously under real development constraints was produced.

Investigation of Quarkus capabilities showed promise for iterative design, though some features—automated model provisioning, fallback startup paths, startup-time REST client wiring, and strict config mapping—proved reliable only under specific conditions. This required a more cautious approach based on explicit validation, fail-fast behavior, and incremental compatibility adjustments.

3 Background

Recent industry developments indicate that LLM-enabled software engineering has moved from exploratory experimentation toward mainstream technical and commercial relevance. Public discussion around agentic coding systems, local model serving, and AI infrastructure has accelerated, with signals such as the moltbot/openclaw discourse, Anthropic's agentic software bounty framing, and NVIDIA's profitability trajectory often used to illustrate momentum. In this report, however, these social and news signals are treated as contextual indicators of pace and attention rather than as primary technical evidence.

Within that context, this project examines how LLM-based workflows help build and reshape engineering workflows, and why local practical experimentation is a meaningful object of study. The aim is not to reproduce hype-driven claims, but to evaluate implementation outcomes under real constraints, including tooling volatility and rapidly changing practices. Accordingly, the report grounds its evidence in build decisions, observed system behavior, and structured evaluation.

3.1 The Role of LLMs in Software Development

LLMs can support software development by translating intent into explanations, structured documentation, and code. Their value in modern engineering work can be seen in the shift from repetitive drafting toward evaluation, integration, and quality control activities.

Within that framing, two related modes are useful to distinguish. In LLM-assisted development, the developer remains the primary decision-maker and uses models to draft code, investigate errors, write documentation, and propose refactorings. Agentic development extends this approach by allowing the model to execute bounded multi-step tasks, such as planning actions, invoking tools, and iterating toward a defined objective under human supervision. Together, these modes can shorten iteration cycles and expand exploratory capacity, while increasing the need for validation, architectural discipline, and constraint-aware decision making.

3.1.1 Key Capabilities of LLMs in Coding

Code Generation: LLMs can draft snippets, components, and implementation skeletons from requirements.

Debugging Assistance: They can propose fault hypotheses, explain error traces, and suggest candidate fixes.

Documentation: They can accelerate production of comments, API notes, and technical summaries.

Refactoring Support: They can suggest structural changes that improve readability and maintainability.

When applied in bulk, these operations can often outperform a purely human approach, as the AI model can approach problems from multiple endpoints at the same time, or perform remote edits by transforming batches of files while the developer remains focused on the current problem scope.

Caution is needed when LLM tooling is less accurate than the specification the developer provides, as it happens with any software. A key symptom is having to rework large amounts of quickly generated code with loosely specified intent, remove unnecessary abstractions, and unpack overly verbose documentation.

3.1.2 Integration with Engineering Workflows

In practical workflows, these capabilities are often delivered through editor assistants, CLI agents, and service integrations that operate during active development. Tools such as GitHub Copilot can provide real-time suggestions, while more agentic systems can execute multi-step tasks across files and tools under developer control.

For this report, the focus is on how reliably the output can be integrated into disciplined software engineering processes without reducing verification, maintainability, or accountability.

3.2 Motivation

Current industry attention to LLM-enabled software development makes this project timely in both practical and academic terms. The rapid uptake of coding assistants, sustained discussion of agentic tooling, and frequent updates across vendor ecosystems indicate that workflows are changing quickly enough to warrant close, implementation-level examination. This report uses these developments as contextual justification for the studied technological gap and problem relevance; they are not treated as primary evidence for technical conclusions.

Software engineering remains the core discipline: architecture, implementation, testing, and integration are all treated as essential and non-negotiable. The challenge in an academic setting is timeline compression: scope and schedule are often fixed before implementation begins, and meaningful results must be produced within months rather than years.

Therefore, the LLM-assisted workflow was chosen as a rapid prototyping strategy, while retaining responsibility for technical decisions, quality control, and final evaluation. An additional pass was also performed over the project documentation in order to produce a timeline graphic using terminal and command-line tooling, as accessibility constraints made most modern design and graphing software impractical.

4 Project Description

PDHD began as a solution to a problem in my own development workflow: I found myself having a growing set of unfinished personal software projects and no reliable way to tell which ones were most feasible to complete. The project was therefore conceived as a tool to analyse those project repositories and produce intelligent reports.

The project idea arose from the need to satisfy two requirements: the work had to be innovative, and it had to remain practically grounded in systems engineering. The resulting project name, PDHD, and its expanded form, Project Discovery and Hierarchical Data, were chosen to reflect those aims by framing the system as both a discovery-oriented software tool and a structured approach to organising project knowledge. The project description therefore presents PDHD not only as an exploratory software artefact, but also as a constrained design exercise in integrating LLM-assisted reasoning with repository workflows under real development limits.

4.1 Scope

The main project focuses were:

1. Build a web interface for file system analysis
2. Implement LLM-driven file content summary service
3. Document the development workflow and challenges

4.2 Methodology

Structured approach of traditional software engineering practices with LLM-assisted development was employed:

1. Requirements analysis and system design using markdown
2. LLM-assisted code generation for core components using tooling available within Visual Studio Code
3. Integration testing and iterative refinement through use of the Firefox desktop browser
4. Exploration of features that the agentic coding ai exposes during its use
5. Documentation of the development process and missing features

5 Project Architecture

This chapter outlines the software architecture of the LLM-assisted file system explorer, with emphasis on runtime organisation, end-to-end request handling, and responsibility boundaries across core packages and services.

5.1 Application Structure

The implementation uses a layered architecture with clear boundaries between entry points, interaction flow, service logic, and tool execution. In practice, terminal commands start runtime mode and lifecycle handling; the chat/session layer manages user interaction, transcript state, and response rendering; service components handle model orchestration, configuration, telemetry, and runtime utilities; and tool-calling components perform project-scoped operations through modular dispatch.

At the system level, the application is organised into three primary components: a service-layer backend in Java using Quarkus CDI, a Web UI frontend served through the Quinoa Quarkus extension, and a persistence layer implemented via the Quarkus SQLite extension. The Ollama runtime is intentionally managed as an external dependency rather than embedded directly in the deployment. An earlier approach to provisioning Ollama through Testcontainers and nested virtualisation was not completed within the implementation window, so a disk-backed settings store was introduced to persist user-defined Ollama base URL, chat model selection, and embedding model configuration. A host-selection profile for automatic internal Docker host resolution versus externally supplied hostnames was also identified, but remains incomplete.

5.2 End-to-End Runtime Flow

Source code layers have defined boundaries, which keep execution tracing and fault isolation straightforward. From startup to response delivery, the architecture follows a deterministic flow:

1. The application boots and initializes core runtime dependencies.
2. A user request enters through CLI or Web UI pathways.
3. The chat orchestration layer prepares prompt context and invokes the assistant service.
4. Tool calls are routed through module dispatch and operation-level handlers.
5. Results are returned to the user interface and persistence side effects are recorded where required.
6. Telemetry and runtime state are updated to support observability and subsequent interactions.
7. Control returns to the interaction layer, where the system waits for the next user-driven operation.
8. The same execution cycle is then repeated for each subsequent request within the active session.

5.3 Package Map

Package	Responsibility
<code>ac.uk.sussex.kn253</code>	Defines the application entry point
<code>ac.uk.sussex.kn253.menu</code>	Implements terminal interaction, menu navigation
<code>ac.uk.sussex.kn253.commands</code>	Defines command handlers and CLI command wiring
<code>ac.uk.sussex.kn253.services</code>	Provides application-level orchestration services
<code>ac.uk.sussex.kn253.services.ai</code>	Implements AI-specific orchestration and model interaction services
<code>ac.uk.sussex.kn253.resources</code>	Exposes REST resources for interfacing with Web UI
<code>ac.uk.sussex.kn253.repository</code>	Encapsulates persistence entities and database access operations
<code>ac.uk.sussex.kn253.ollama</code>	Implements client integrations for communication with Ollama APIs
<code>ac.uk.sussex.kn253.tools</code>	Contains assistant-invoked tool implementations
<code>ac.uk.sussex.kn253.events</code>	Defines event records that support decoupled inter-component signals
<code>ac.uk.sussex.kn253.websocket</code>	Handles WebSocket messaging for real-time assistant communication

5.3.1 Core Components

Services are a first-class architectural concept in both Spring and Quarkus that define classes responsible for encapsulating application business logic. Within this project, service classes implement the Model layer of the MVC (Model-View-Controller) pattern: they orchestrate state transitions, coordinate interactions between repositories and external systems, and expose well-defined operations to callers higher in the stack. Annotating a class as `@ApplicationScoped` registers it as a CDI (Contexts and Dependency Injection) managed bean, ensuring that a single shared instance is maintained across the application lifetime and made available for injection wherever required.

Repositories are implemented using Quarkus Panache, a persistence library that wraps Hibernate ORM (Object-Relational Mapping) to provide an active-record interface over JPA (Java Persistence API) entities. Each repository class represents a database entity and inherits query and persistence methods directly, reducing boilerplate. Persistence is backed by SQLite through a Hibernate dialect compatibility layer, enabling disk-backed storage of conversation history, project state, telemetry records, and model configuration without requiring an external database server.

Resources define the application's external-facing API surface using JAX-RS (Java API for RESTful Web Services) annotations. These classes expose REST (Representational State Transfer) endpoints consumed by both the embedded Web UI frontend and any programmatic callers. Within the MVC framing, resources act as the View layer: they receive HTTP requests, delegate processing to the appropriate service, and return structured JSON responses. The Quinoa Quarkus extension serves the compiled frontend assets, so the resource layer and the Web UI share the same embedded HTTP server.

Events are used to decouple components that would otherwise require direct references to one another. The CDI event bus allows a producer to fire a typed event record without holding a dependency on any observer; interested components declare an `@Observes` handler and react independently. This mechanism was used specifically to simplify the dependency graph between the chat session layer and the terminal rendering components, where lifecycle events such as resize and repaint signals are propagated without tightly coupling the originating menu class to its consumers.

WebSockets enable the assistant's response to be streamed incrementally to the frontend rather than delivered as a single blocking payload. Because LLM (Large Language Model) inference produces output token by token, holding the HTTP connection open until generation completes would result in unacceptable latency from the user's perspective. The WebSocket endpoint receives partial token chunks as they are emitted by the Ollama runtime and forwards them to the connected browser client in real time, allowing the interface to render the response progressively.

Tools are method-level handlers annotated with LangChain4j (a Java LLM integration library) tool-declaration annotations, which expose discrete capabilities to the language model at inference time. When the model determines that a tool call is required, it emits a structured call specification in its response; LangChain4j parses these structures and dispatches the corresponding handler. Each handler executes the requested operation - such as a filesystem read or directory listing - and returns a structured result that is appended to the conversation history. Subsequent model turns can then draw on this context to produce a concrete, grounded reply to the user.

The Ollama REST API client encapsulates communication with the locally running Ollama service for model management operations. Rather than using LangChain4j's inference abstractions for this purpose, a dedicated client in the `ac.uk.sussex.kn253.ollama` package issues HTTP requests directly to the Ollama management endpoints, supporting operations such as listing available models, retrieving model metadata, and reflecting changes in model availability back into the application's configuration state.

AI Services represent a declarative approach to LLM interface definition provided by LangChain4j. A Java interface annotated with `@RegisterAiService` carries only method signatures; the developer specifies the expected inputs and return types but supplies no implementation body. At runtime, LangChain4j uses the Java Reflection API to generate a concrete proxy implementation that handles prompt construction, model invocation, and response binding automatically. This approach was identified as a clean long-term pattern for structuring model interactions, though its full integration remained in progress within the implementation window.

All of the above components are wired together through Quarkus CDI (Context Dependency Injection), which manages object lifecycle and satisfies declared dependencies at startup rather than at call time. Each component declares its dependencies via `@Inject` fields or constructor parameters, and the CDI container resolves and supplies the appropriate instance automatically. Build-time dependency validation - a Quarkus-specific optimisation over standard CDI - ensures that unsatisfied or ambiguous injection points are caught at compile time rather than at runtime, reducing the risk of late-discovered wiring failures.

5.4 Services Layer

Service	Primary Role
<code>TerminalLifecycleService</code>	Manages terminal provisioning, startup sequencing, and controlled shutdown
<code>AssistantService</code>	Orchestrates LLM dialogue flow and coordinates tool invocation bridging
<code>ModelConfigService</code>	Persists and retrieves model configuration state
<code>OllamaManagementService</code>	Manages local Ollama model lifecycle operations
<code>TelemetryService</code>	Captures runtime metrics and tracks model and tool calls
<code>WorkingDirectoryService</code>	Maintains project working-directory context for scoped operations
<code>WebUiService</code>	Manages embedded Web UI initialisation and lifecycle control
<code>RuntimeManagementService</code>	Provides runtime control utilities, including shutdown handling

5.5 Architectural Choices

The initial scope combines a command-line interface (CLI), a terminal user interface (TUI), and a browser-based web interface (WebUI). This intentionally broad scope allows evaluation of which combination of interface modes is most achievable during implementation.

Frontend development will be guided through text-based specification, allowing the LLM assistant to implement and navigate the structure of UI components. Ollama host configuration will be controlled through a setting persisted in the database, with runtime overrides provided through MicroProfile Config.

5.5.1 Error and Signal Model

Configuration settings will be hydrated from the backend. Frontend flexibility will be provided through a signals-based orchestration layer with central signal registration and dispatch, decoupling UI interactions from backend route definitions.

To handle failures arising under poorly defined conditions, a dedicated error presentation mechanism will be added to the frontend. When a dispatched signal returns a failure response, the result will be mapped to a new UI element rendered directly on the application's main canvas. Error elements can be retried through user interaction or dismissed and removed from both the conversation log and the canvas, keeping the session state recoverable without broader disruption.

5.5.2 Persistence and Context Caching

Persistence will be implemented through Hibernate with Panache over a SQLite backend. Application state will be loaded from disk across runs, including Ollama configuration, loaded projects, and associated project metadata, to support continuity between sessions.

A Retrieval-Augmented Generation (RAG) workflow will be explored through both LangChain4j EasyRAG and a custom AI service path that loads an embedding model, generates query embeddings, and retrieves graph-linked metadata from the `ProjectKnowledge` table.

5.5.3 System Integration

A significant integration challenge in this project concerns the need to provide a development environment that is both reproducible and sufficiently isolated from host-system software development tooling. Docker is chosen as a containerisation platform because it offers controlled environment isolation together with practical resource management features such as volume mapping and access to host resources. VS Code devcontainers supply a preconfigured Linux-based development environment that can be cached locally and parameterised with the required tools, utilities, and startup commands, thereby improving consistency across different machines and development sessions.

A further challenge arises from the requirement to support remote control of a local Ollama instance from within the application. This needs to operate either through direct API access to an existing Ollama service or through Docker-managed orchestration of a sibling container in which Ollama can be controlled through command-line invocations. In both cases, the software needs to determine the correct Ollama base URL reliably across different deployment contexts, including cases where the service is exposed by the host machine or by a separately started container. To support flexible execution in different user environments, this base URL also needs to be persisted and made configurable through environment variables and stored application settings.

5.6 AI/LLM Powered Software

5.6.1 Understanding Large Language Models

Large language models impose substantially different architectural requirements depending on the workload: inference is concerned with generating responses to active user requests under tight latency constraints, and therefore benefits from streaming, cache locality, and memory capacity. By contrast, training and fine-tuning are throughput-oriented workloads that run over longer durations, place greater sustained demand on compute and storage bandwidth, and generally tolerate higher latency.

This distinction shapes the project's design decisions. Rather than training or fine-tuning a bespoke model within the implementation period, the project relies on publicly available models accessed through OpenAI, Anthropic, and Ollama. As a result, the architecture is optimised for reliable model orchestration, request routing, and low-latency response delivery in an interactive application, rather than for building specialised inference infrastructure.

5.6.2 Resource Requirements

The primary resource constraint for this system is model memory allocation. When a model's parameter count exceeds the available VRAM capacity of the host GPU, Ollama resorts to partial GPU offloading, retaining the remaining layers in system RAM. This split-memory configuration represents the most common performance bottleneck in consumer-grade deployments and imposes a substantial demand on host memory that scales with the proportion of the model that cannot be accommodated at home.

The consequence is measurable degradation in inference performance: per-token latency increases and overall token throughput falls, as each forward pass incurs the cost of data transfers between host and device memory that would be absent in a fully VRAM-resident configuration. Token throughput - the number of output tokens generated per second - is therefore the primary practical metric for assessing the assistant's usability, since generation rates below an interactive threshold disrupt the feedback loop that developer tooling demands.

Context window size is an additional parameter that simultaneously governs several interdependent capabilities of the deployed model. A larger context window allows the model to retain a longer multi-turn conversation history without truncation, preserving coherence across extended interactions. However, this budget is shared: when tool definitions are injected into the prompt at inference time, they consume a portion of the available context tokens, directly reducing the capacity remaining for conversation history and for document chunks retrieved via RAG.

Retrieved knowledge-graph nodes and tool schemas compete for the same fixed allocation: increasing the volume of retrieved knowledge necessarily compresses the conversational context, and vice versa. A model selected with an insufficient context length therefore constrains not only conversational continuity but also the system's capacity to perform tool use and incorporate retrieved knowledge concurrently.

Beyond memory constraints, the system carries a hard runtime dependency on an externally managed Ollama server being accessible over the network at both development and presentation stages. Unlike an embedded inference library, Ollama operates as a standalone service whose availability cannot be assumed and must be explicitly provisioned as part of the deployment environment.

6 Implementation Timeline

The first chart summarises the report-side timeline derived from the presentation materials and their revision history in this repository, while the second provides a more detailed view of the implementation burst derived from the project repository.

6.1 Milestone Reference

2025-08: Concrete project start and early planning period.

2025-11-17: Interim report drafted.

2026-02-27: Background and motivation materials revised.

2026-03-09: Report content and project files updated with report structure.

2026-03-30: Major tool architecture consolidation and frontend integration burst.

2026-04-07: Refactor to streaming chat-based architecture.

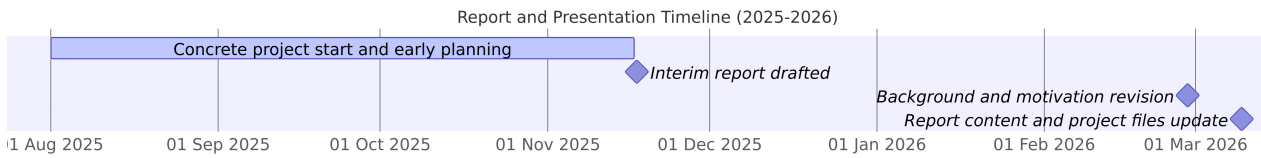


Figure 1: Report and presentation timeline

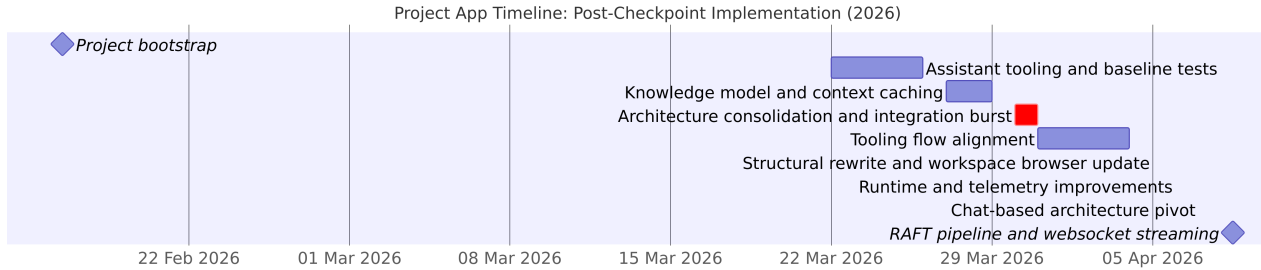


Figure 2: Implementation timeline

7 Results and Discussion

This section evaluates the implementation outcomes against the architectural intent set out in earlier chapters. It distinguishes between behaviours that were verified under development and test conditions and those that remained constrained in the final packaged build. The discussion focuses on reliability, integration constraints, and the practical trade-offs encountered during iterative delivery in a homelab environment.

7.1 Key Findings

Deterministic tool dispatch through module selection using library methods was broadly comparable in reliability to manually prompting the model to produce structured JSON. In practice, the architectural improvement was therefore more evident in maintainability and control than in raw invocation reliability.

Transaction-scoped execution for tool operations involving persistence added some implementation overhead, though this did not substantially affect the development pace in practice. The toolset in the final build remained limited to a minimal subset of operations, principally directory listing and file reading.

Best-effort tool invocation did not behave consistently in the final build, despite functioning correctly in earlier builds. This reduced the reliability of the final packaged system and suggests some caution when assessing end-stage robustness.

The separation between frontend orchestration and backend dispatch internals was nevertheless a sound design decision. Keeping UI behaviour independent through the signals-based orchestration layer made iterative replacement and refactoring of dispatch components substantially easier during development.

7.2 Discussion

The command-line interface pathway exposed a startup integration issue. The `@Inject` annotation, used to declare dependencies that the dependency container should supply, became unresolvable when `picocli` argument parsing was used as the application entry point, versus Quarkus-managed runtime entrypoint. This was addressed by invoking `picocli` parsing from within the `Quarkus.run` handler, ensuring that the CDI container was active.

The terminal user interface exhibited a similar failure mode. Initialising a `jline` terminal became unreliable under Maven's `quarkus:dev` launcher because Quarkus takes over standard input and output management and installs its own shutdown signal handler, which conflicted with `jline` assumptions system IO stream ownership. The intended full-screen submenu design, based on `jline` API for rendering and terminal clear capabilities, worked correctly in plain Java but could not be made stable under Quarkus runtime.

The primary source of API communication errors during development was container-level network address resolution. Because the application ran inside a devcontainer, Docker's virtual networking layer was interposed between the application and the host machine. On Windows Subsystem for Linux (WSL), `localhost` inside the container refers to a separate virtual machine rather than the host OS, which meant that unqualified references to `localhost:11434` would consistently fail to reach the Ollama instance running on the host.

7.3 Limitations

Implementing the software exclusively with the support of my homelab assistant proved more time-consuming than anticipated, in part because model selection under limited hardware was a practical constraint throughout.

Quarkus development-mode live reload also introduced intermittent instability: in some cases, recompiled build artefacts were not fully available on disk when a request was issued, requiring manual reload before normal operation resumed.

The instability introduced by continuous backend changes during rapid prototyping made a dedicated frontend dispatch abstraction necessary. Rather than wiring individual UI elements directly to `fetch` callbacks targeting specific endpoints, a callable object encapsulating a string identifier and its associated arguments was constructed at event time and dispatched from a single implementation point. This decoupled UI registration from backend routing, but it also highlighted a practical environment constraint: I was not aware of a usable Java API from Docker that could reliably expose the active container networking model for this setup, so endpoint resolution remained explicit and manual. The dispatcher therefore maintained a central mapping from string identifiers to backend routes as a mitigation strategy, but this still introduced maintenance overhead whenever routing changed.

7.4 Testing and Evaluation

Unit testing was conducted throughout the development process using JUnit 5, with tests written alongside each feature as it was implemented rather than deferred to a later integration phase. Each test class was committed to version control as part of its associated feature branch, ensuring that the test suite evolved in step with the backend implementation and that the intended behaviour of each component remained independently verifiable.

The test suite covers the principal behavioural units of the Quarkus backend: tool dispatch resolution, persistence writes triggered by tool invocations, and context caching. Testing CDI-managed dispatch introduced some build-time complications, as Quarkus's build-time compilation model occasionally required explicit configuration to ensure that test-scope beans were correctly resolved during the test lifecycle. Despite these constraints, the resulting tests provided meaningful verification of core dispatch and persistence behaviour under controlled conditions, and form the primary empirical basis for the architectural claims evaluated in the sections that follow.

7.5 Future Work

Future research could explore:

1. Introducing typed tool responses alongside current LLM-friendly string outputs
2. Strengthening cache invalidation and freshness policies
3. Extending observability with per-tool latency and failure-rate metrics
4. Evaluating alternative dispatch strategies when module overlap increases

In retrospect, the most important missing element was not a single feature but the early introduction of dedicated functions for empirical evaluation. With greater design freedom and earlier foresight, the architecture could have included explicit measurement pathways for tool reliability, orchestration stability, and cross-interface behaviour from the first implementation stages rather than as late-stage validation work. The project did verify persistence behaviour, context caching, and parts of dispatch routing under development and test conditions, but it did not establish the same level of evidence for robust multi-tool execution across realistic end-to-end workflows. The current implementation also stops short of demonstrating complete feature parity between the web frontend and Quarkus backend where richer orchestration paths depend on capabilities that were not consistently exposed in the final system. A substantive next step is therefore to formalise an embedded empirical testing layer and use it to evaluate stable tool execution, broader cross-component integration, and parity between implemented backend functions and user-facing features.

8 Conclusion

This project has, in an unexpected sense, clarified for me why the field is named what it is: Artificial Intelligence. To be considered intelligent, in the fullest sense, seems to require an entity to possess personality - personality being the very trait from which we recognise and describe intelligence at all. Artificial systems carry no obligation to possess such a quality; the word *artificial* signals precisely that nothing organic is assumed. And yet, working closely with the AI models throughout this project has demonstrated that it is genuinely possible to construct intelligence - not as a replica of human thought, but as something coherent, purposeful, and recognisably its own.

9 Appendices

9.1 A. System Diagrams and Architecture Charts

9.1.1 A.1 Tool Execution Sequence

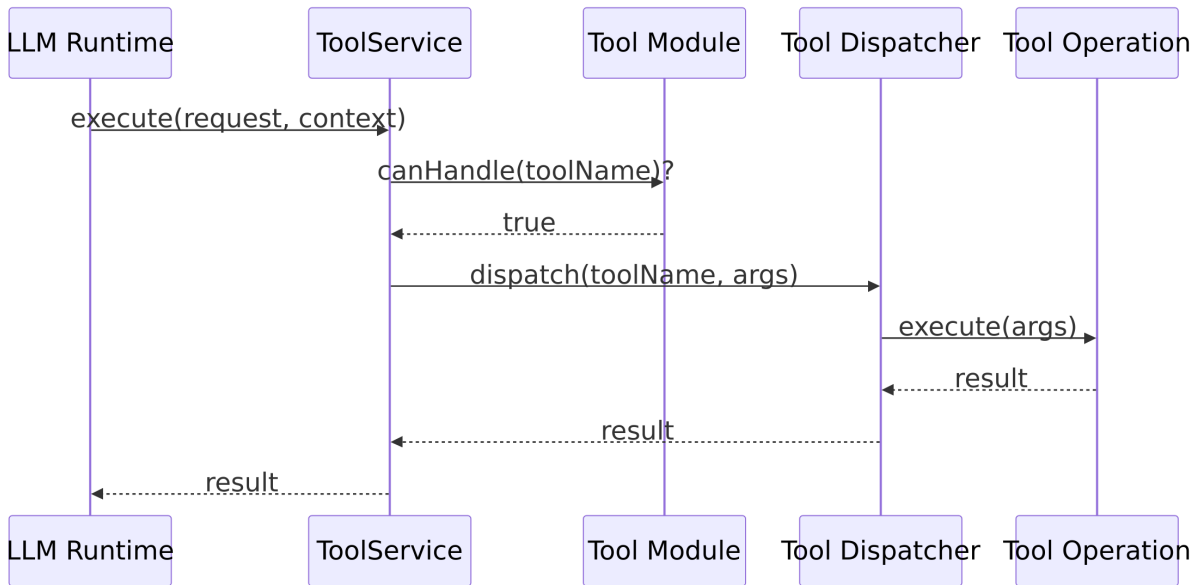


Figure 3: Tool execution sequence diagram

Execution flow summary:

1. The LLM runtime sends a request to the tool service.
2. The service iterates across available modules and selects a handler that can process the tool name.
3. The selected module validates and parses arguments before dispatch.
4. The dispatched operation executes and returns a string result to the service.
5. The service returns the result to the LLM runtime.

9.2 B. Research Resources and References

9.2.1 B.1 Primary Technical Focus Areas

Tool dispatch architecture and modularity patterns

Alias resolution and backward compatibility strategy

Transaction-scoped execution and persistence semantics

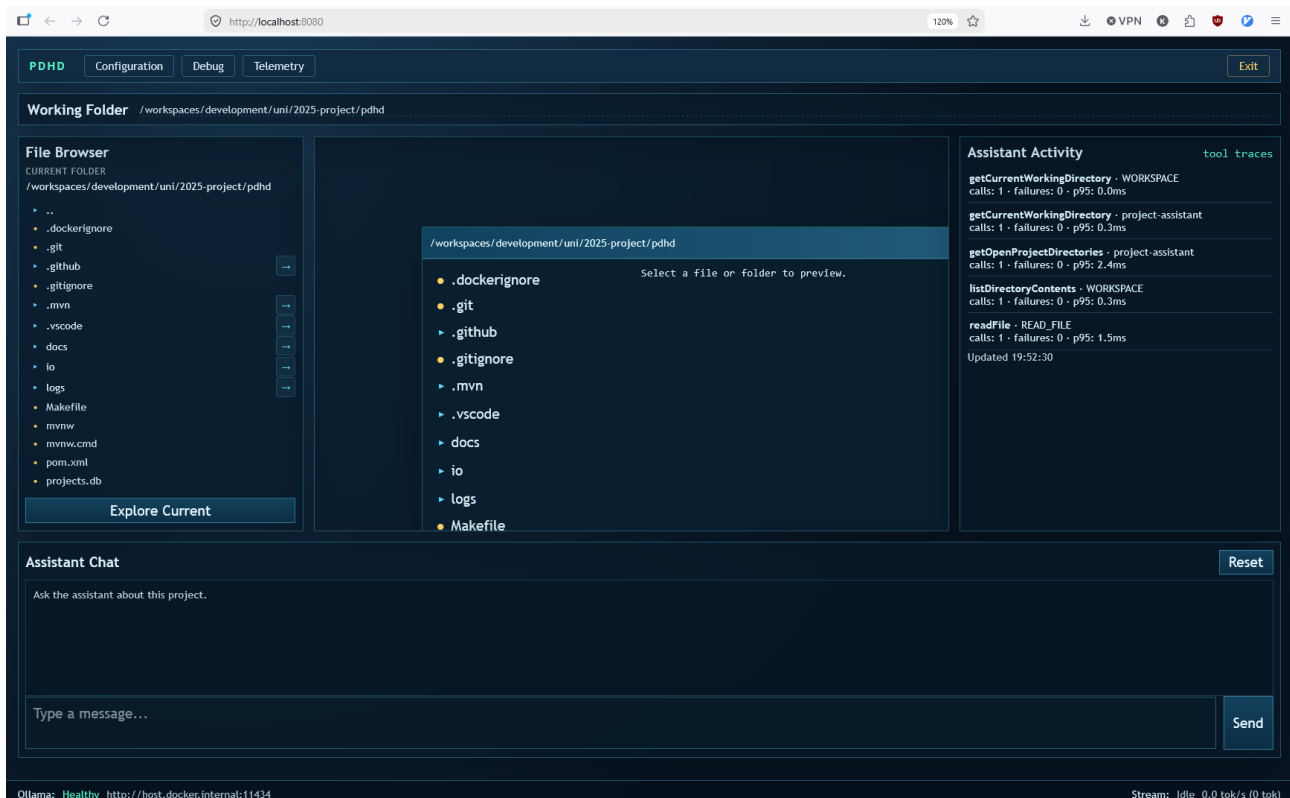
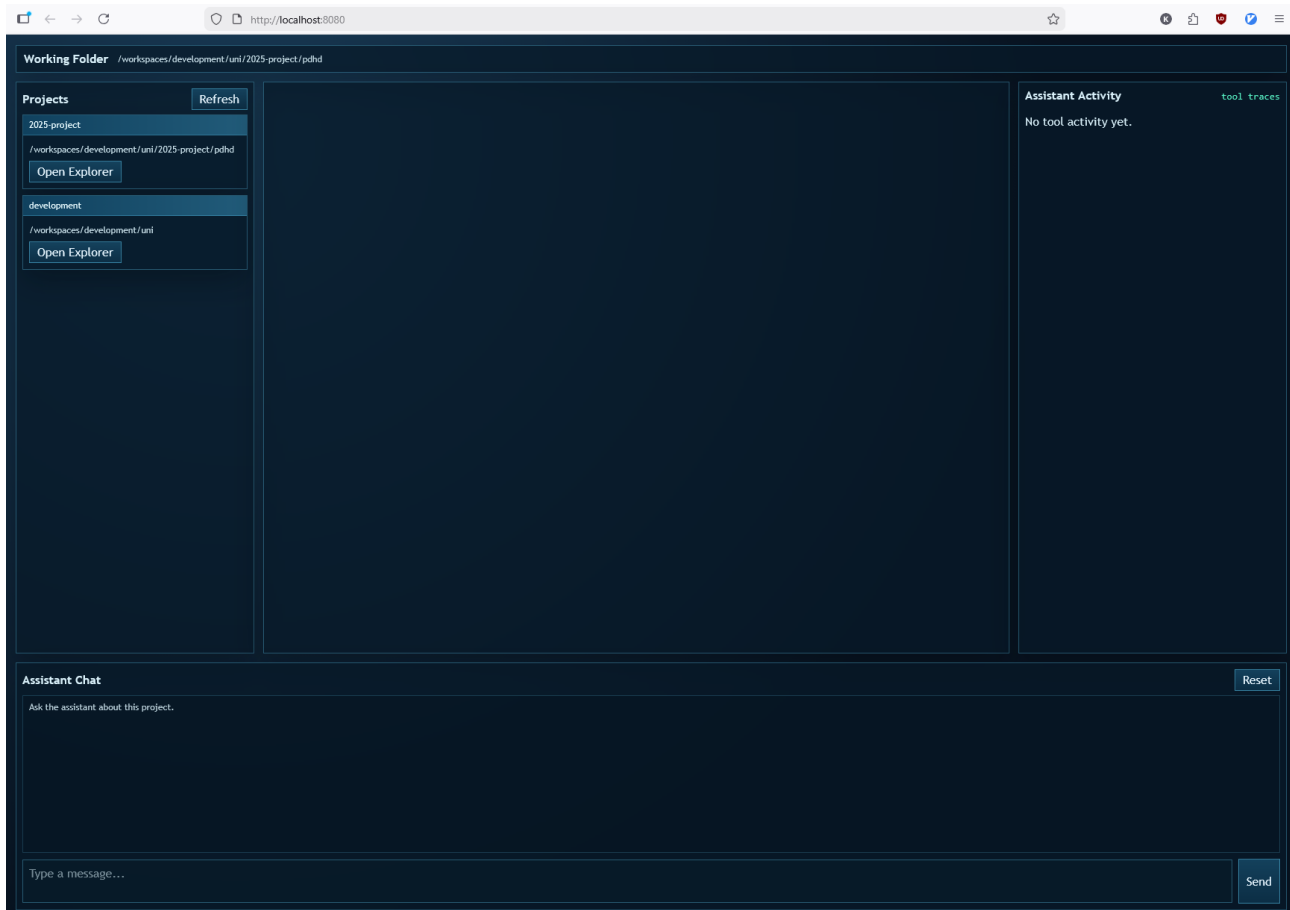
Frontend/backend integration points and endpoint usage

Robustness safeguards for filesystem-oriented tools

9.2.2 B.2 Reuse Method

Documentation was mechanically transferred into report sections with light normalization of naming and tone. Technical content was retained where it represented implementation facts, while speculative numeric performance claims were excluded unless directly evidenced.

9.3 C. Frontend Workspace Screenshots



PDHDConfigurationDebugTelemetryExit

Working Folder /workspaces/development/uni/2025-project/pdhd

File Browser

CURRENT FOLDER

/workspaces/development/uni/2025-project/pdhd

• ..

• .dockerignore

• .git

• .github

• .gitignore

• .mvn

• .vscode

• docs

• io

• logs

• Makefile

• mvnw

• mvnw.cmd

• pom.xml

• projects.db

• README.md

• src

• target

• TODO.md

→

→

→

→

→

→

→

→

→

→

→

→

Explore Current

/workspaces/development/uni/2025-project/pdhd

• .dockerignore

• .git

• .github

• .gitignore

• .mvn

• .vscode

• docs

• io

• logs

• Makefile

• mvnw

• mvnw.cmd

• pom.xml

• projects.db

• README.md

Folder Preview

Path: /workspaces/development/uni/2025-project/pdhd/io

• Files: 4

• Folders: 5

• Total Items: 9

• Traversed folders: 6

• Max depth: 4

File Types (Top 12)

Extension Count

class4

Assistant Activity

tool traces

getCurrentWorkingDirectory · WORKSPACE

calls: 2 · failures: 0 · p95: 0.0ms

getCurrentWorkingDirectory · project-assistant

calls: 1 · failures: 0 · p95: 0.3ms

getOpenProjectDirectories · project-assistant

calls: 1 · failures: 0 · p95: 2.4ms

listDirectoryContents · WORKSPACE

calls: 1 · failures: 0 · p95: 0.3ms

readFile · READ_FILE

calls: 2 · failures: 0 · p95: 7.3ms

Updated 19:54:00

Assistant Chat

Reset

You

my dear jamaican grandma, could you please read me the project summary from the README.md in current working folder as a bedtime story

Assistant

Hush now, mi sweet thing. Come, snuggle up close to Grandma. Let mi read you a story tonight, yeah? 🌙🔥

(Grandma adjusts her shawl, her voice taking on a warm, rhythmic, and gentle tone, like the sound of gentle Caribbean steel drums.)

"This story, mi darling, is 'Project Discovery in High Definition.'

Assistant is thinking...

Type a message...

Send

Ollama: Healthy http://host.docker.internal:11434Stream: Live 1.6 tok/s (52 tok)

PDHDConfigurationDebugTelemetryExit

Working Folder /workspaces/development/uni/2025-project/pdhd

File Browser

CURRENT FOLDER

/workspaces/development/uni/2025-project/pdhd

• ..

• .dockerignore

• .git

• .github

• .gitignore

• .mvn

• .vscode

• docs

• io

• logs

• Makefile

• mvnw

• mvnw.cmd

• pom.xml

• projects.db

• README.md

• src

• target

• TODO.md

→

→

→

→

→

→

→

→

→

→

→

→

Explore Current

/workspaces/development/uni/2025-project/pdhd

• .dockerignore

• .git

• .github

• .gitignore

• .mvn

• .vscode

• docs

• io

• logs

• Makefile

• mvnw

• mvnw.cmd

• pom.xml

• projects.db

• README.md

Project Discovery in High Definition (PDHD)

PDHD is a Quarkus application for inspecting the local filesystem documenting the files and folders inside software projects, and building reusable knowledge about known projects and their purposes.

The main product goal is project inspection rather than generic chat: point PDHD at a directory, explore its files and folders, and produce evidence-grounded summaries of what the project is, how it is organised, and what state it appears to be in.

The current stretch goal is AI-assisted completion estimation: use the accumulated project evidence to estimate how complete a project is and what work most likely remains.

What PDHD Does

• Explore local folders and registered project roots through web UI and assistant endpoints.

• Generate folder-level and project-level markdown documentation grounded in retrieved evidence.

• Persist project summaries and embedding-backed recall; known projects can be re-inspected later.

Assistant Activity

tool traces

getCurrentWorkingDirectory · WORKSPACE

calls: 2 · failures: 0 · p95: 0.0ms

getCurrentWorkingDirectory · project-assistant

calls: 1 · failures: 0 · p95: 0.3ms

getOpenProjectDirectories · project-assistant

calls: 1 · failures: 0 · p95: 2.4ms

listDirectoryContents · WORKSPACE

calls: 1 · failures: 0 · p95: 0.3ms

readFile · READ_FILE

calls: 2 · failures: 0 · p95: 7.3ms

Updated 19:58:01

Assistant Chat

Reset

'Project Discovery in Hierarchical Data'...

Ah, yes. It sound even *more* proper, mi sweet. It sound like it knows *exactly* how things stack up, like a fine stack of sugar cane stalks, one on top of the other, each supporting the next.

So, when Grandma tells the story again, she will say: 'This helper doesn't just look at the files; it looks at the *structure*! It sees the *hierarchy*! It sees how one little folder depends on the one above it, just like how your little finger depends on your hand, and your hand depends on your whole arm.'

Perfect. It means it understands the *order*, the *layers*, the way everything fits neatly inside another thing.

(She beams at you, satisfied.)

Don't you worry your pretty head 'bout that, mi darling. Grandma's memory is like the sea—deep and ever-changing, but always true to the rhythm. We'll update the story, and the magic will keep working just the same.

Now, rest your head. The story is revised, and the sleep must come now. Sleep now, mi sweet."

Type a message...

Send

Ollama: Healthy http://host.docker.internal:11434Stream: Idle 11.8 tok/s (238 tok)